CS [46]290 ECE [46]100
Conte

Memory Hierarchies, Cache Memories

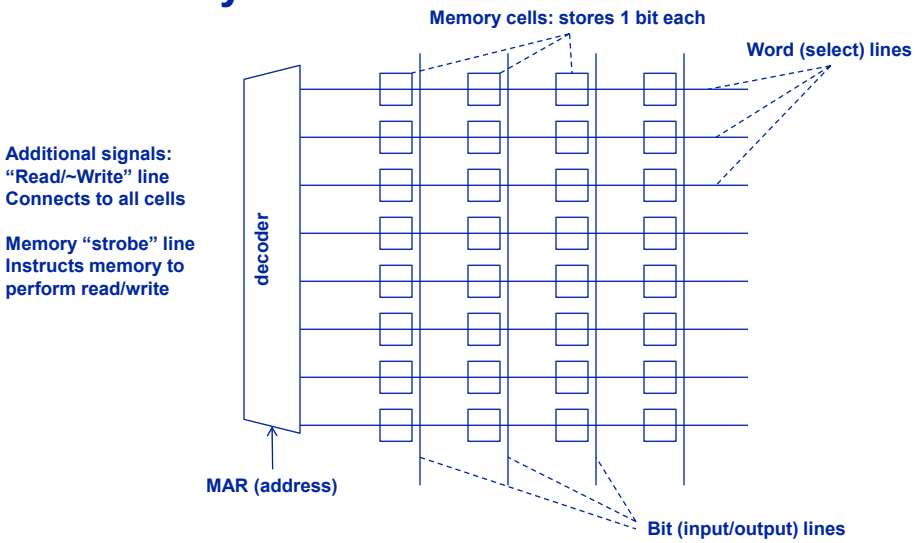
Last revised Spring 2019

Copyright © Thomas M. Conte

Random Access Memory

- "Random" means access time is "the same" for all locations
  - As opposed to sequential access, as in tapes, disk
- SRAM: Static Random Access Memory
  - Low density, high power, $expensive$, fast
  - "Two inverters arguing, you only trust one of them"
  - Static: content will last "forever"(until lose power)
  - SRAM cell about 10X larger than DRAM cell
  - Burns A LOT more power than DRAM
- DRAM: Dynamic Random Access Memory
  - High density, low power, cheap, slow
  - FORGETS– "dynamic" is marketing: need to be "refreshed" regularly- memory that forgets!

Structure of a random access memory



GENERAL TRUTH

- Memory is either FAST *or* CHEAP, but never both
- Trick is to use speculation and prediction to make a combined memory that *behaves as if it were* both FAST and CHEAP

Small, fast memory ("Cache")

Use speculation with prediction to fill the cache so that the CPU "sees" fast memory performance most of the time

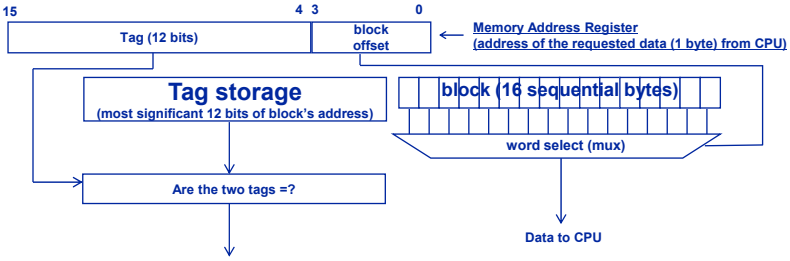
Main memory: slow, cheap

Spatial and Sequential Locality

- **Predicting program behavior:**

- **Spatial locality**
  - Items with addresses near one another in memory are likely to be referenced close together in time
  - If access ***X*** at time ***t***, then highly likely also access $X \pm A$ at time $t \pm D$ for small ***A, D***
- **Sequential locality**
  - Important special case of spatial locality
  - next item accessed is likely to be at the next sequential address in memory
  - If access ***X*** at time ***t***, then also access $X + B$ at time $t + 1$ for a constant **B**

Simple, 1-block cache



If the tag from the address and the tag of the block in the cache match, then the data output of the word select mux is the right data

Called a "HIT"
Takes time = "Hit Time"

If however the tags do not match, it's the wrong data:

(1) Get block of 16 bytes of data from main memory
(2) Overwrite current block with new block, overwrite tag with new tag
(3) Give data to CPU

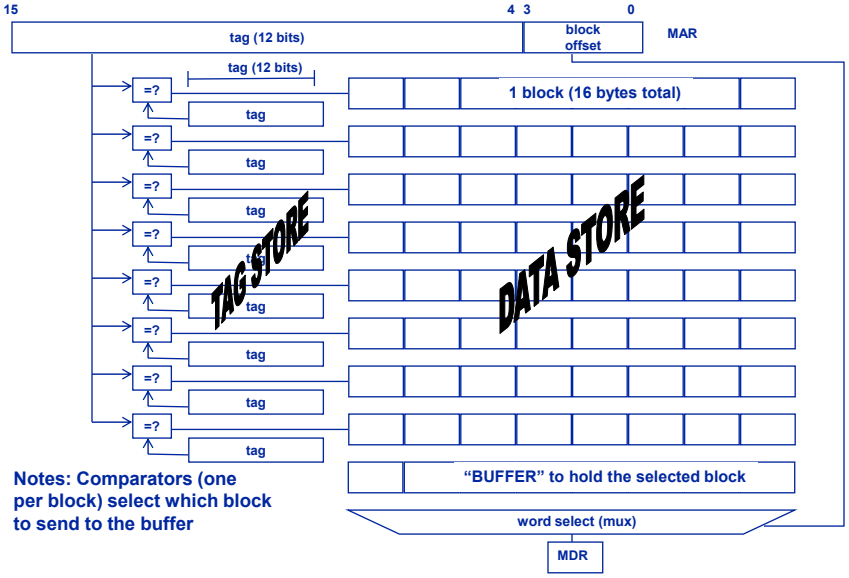
Called a "MISS"

This process is called "MISS repair"
Takes time = "Miss Penalty"

Temporal locality

- Recently accessed items are likely to be re-accessed in the near future
- If access *X* at time *t*, then highly likely also access *X* at time $t \pm D$ for small *D*

Storing multiple blocks: 128 byte Fully Associative Cache

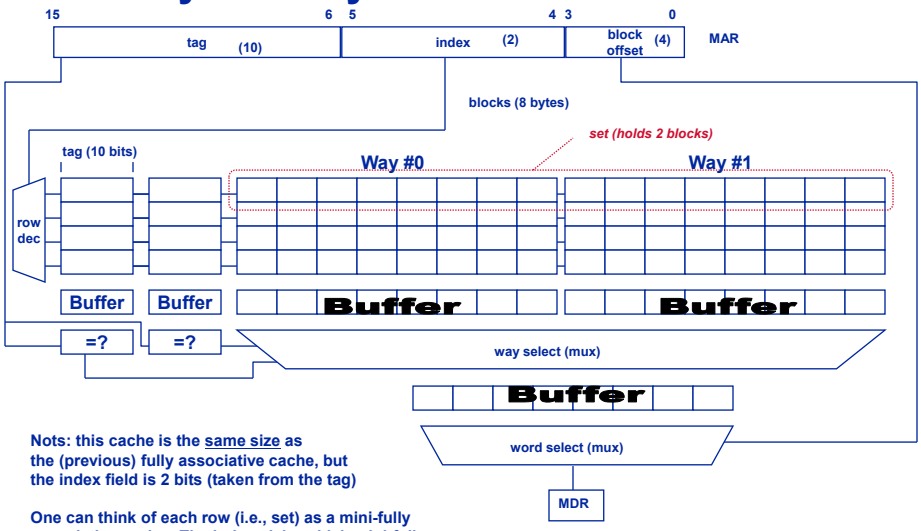


Notes: Comparators (one per block) select which block to send to the buffer

Problems with a Fully Associative Cache

- Searching the 8 blocks takes either TIME or, if it is done in parallel, wastes POWER
  - Use a simpler cache– see direct-mapped caches (below)
- If the cache is full of blocks and the CPU asks for a block not in the cache, which block is kicked out ("replaced?")
  - Will discuss this in a little bit

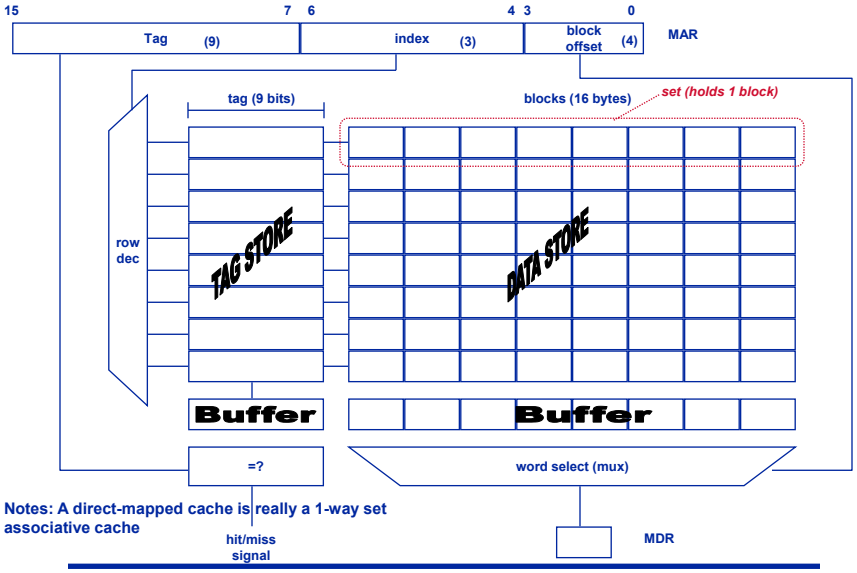
A 128byte 2-way set associative cache



Nots: this cache is the same size as the (previous) fully associative cache, but the index field is 2 bits (taken from the tag)

One can think of each row (i.e., set) as a mini-fully associative cache. The index picks which mini-fully associative cache to search

A 128 byte direct mapped cache



Direct-mapped cache discussion

- Advantages: simpler, no need to search
- Disadvantages: ridged placement of block to a given row means two blocks can conflict and replace each other– bad if they are both used in the immediate future by the CPU
- Set associative cache is a hybrid approach between Direct-mapped and fully associative

Cache dimensions

(C, B, S)

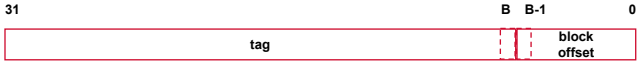
2^C = total number of bytes in the data store of the cache

2^B = total number of bytes per each block

2^S = total number of blocks per set

Decoding the address

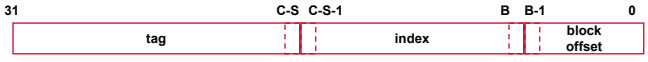
- Fully associative, 2^B-byte blocks



- Direct mapped cache size 2^C



- Set associative, sets of size 2^S blocks per set



- In general, the 3-tuple (*C*, *B*, *S*) completely specifies the dimensions of a cache if the set associativity is a power of two

Cache performance and AAT

Access time for a hit = Hit time

Access time for a miss = Hit time + Miss penalty

Average access time = Hit time + (Probability of a miss) × (Miss penalty)

Probability of a miss = "Miss ratio"

To measure miss ratio:

(1) Run program and measure total # misses / total # accesses

(Problem: Need to build the computer hardware first!)

Or

(2) Simulate the cache running a program and messure miss ratio

Example

Hit time = 2 cycles

Miss rate = 0.05

Miss penalty = 11

Average access time = 2 + 0.05 × 11

= 2.55 cycles

Summary: the meaning of cache performance terms

- Hit: data is in the cache
  - Hit Time: Time to get the data from the cache to the processor
- Miss: data is not in the cache
  - Miss Ratio = number of misses / total number of accesses
  - Miss Penalty: Time to get a missing block and get the data to the processor

Miss penalty (MP) calculation

- Memory has at least three performance parameters:
  - Size of a transfer (in bytes) (i.e., the bus width in bytes)
  - Latency for first transfer (in CPU cycles)
  - Latency for subsequent transfers
  - Oftentimes expressed as (Latency for first word):(Latency for subsequent words), e.g, "10:1" memory

$$\text{Miss penalty} = \text{Latency for first xfer} + \left(\frac{2^B}{\text{Xfer size}} - 1 \right) \times (\text{Latency for subsequent xfers})$$

Example

$$\text{Latency for first xfer} = 10 \text{ cycles}$$
$$\text{Latency for subsequent xfers} = 1 \text{ cycle}$$
$$\text{Transfer size} = 4 \text{ bytes}$$
$$B = 3$$
$$\text{Miss penalty} = 10 + \frac{8}{4} - 1 = 11$$

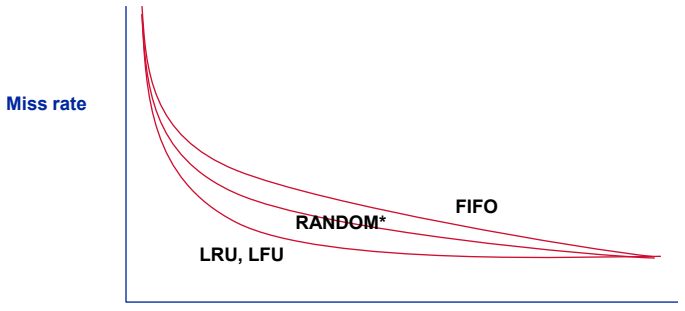
Cache design policies

- Block placement
  - Where to place a block in the cache -- We "always" use bit selection based on address (in this class, but other approaches have been proposed)
- Block replacement
  - Which block to replace on a miss
- Write strategy
  - What happens on a write

Block Replacement

- Consider a 4-block fully-associative cache:
  - Trace: block A, B, C, D, D, D, B, C, A, E
  - Which block should be replaced to make room for E?
- Least likely to use in the future
  - Problem: need to know the future!
- Least recently used (LRU):
  - Replace block D
  - Idea: LRU block is least likely to be used again
- Others:
  - Random:
    - Replace any of A, B, C, D with equal probability
  - Least frequently used
    - Replace A, B or C
  - First in, first out
    - Replace A (in general not the same as LRU)

Typical performance of block replacement algorithms



(*you might get lucky with RANDOM)

LRU is a "stacking algorithm"

- You can view the relative ranking of blocks in an LRU cache as a special kind of stack
  - New reference: PUSH the block address

	A	B	X	D	F
Stack:	A	B	X	F	
		A	B	X	
			A	B	
				A	

  - Old reference: REPUSH the block address

Stack:	B	F	…
	F	B	F
	X	F	B
	B	X	X
	A	A	A

LRU stack as a cache simulator

- The LRU stack simulates all fully-associative caches from size = one block to infinity (or the size of the maximum stack-depth)

  Rule: If a block is found at depth *d* during a REPUSH, then
  - the block is a MISS in all caches with size < *d* blocks, and
  - the block is a HIT in all caches with size ≥ *d* blocks

Cache write strategy

- Writes are rarer than reads
- about 5-7% of all accesses, and 1/4th of all data accesses
- Writes should be efficient
- Two aspects:
  1. When to write, and
  2. Whether to allocate on a miss.

Write strategies (cont.)

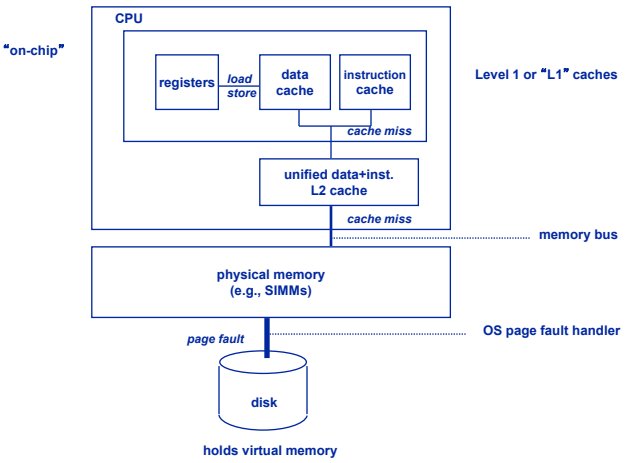
1. When to write:
   - Write back
     - Only write to memory when a block is replaced
     - Need "dirty bit" to mark when a block is written
   - Write through
     - concurrently write to cache and memory
     - easy to implement, no "dirty bit" overhead
     - Problem: high memory traffic
2. Whether to allocate on a miss:
   - Write allocate
     - bring block into cache on a miss
   - Write no allocate
     - Don't bring block into cache on a miss.

- Combinations:
  - Write back often combined with write-allocate
  - Write-through often combined with write-no-allocate

Two cache access approaches

- Hit time calculation depends on this
- Tag first, Data second
  - Advantage: low power/low energy
  - BUT will have a higher hit time (HT)
  - Generally only used in low power/low energy environments. Often keep associativity very high to avoid misses
- Tag and Data in Parallel (what we showed)
  - Advantages: fast, data access time is "hidden" in the hit time (HT)
  - ***This is what we will focus on in this class***

Typical Memory Hierarchy



INTERLUDE: WHAT IS VIRTUAL MEMORY?

Virtual Memory vs. Caching

- Virtual Memory Systems implement a software cache
  - Traditionally, each page in virtual memory is given an entry in a "page table"
    - This is not the same as a cache tag store!
    - Analogy to caches: every block-sized range in memory would have an entry in a "block table"
  - Today, some systems used "inverted page tables," which are (very) similar to caches
    - One entry per page in main memory
    - Holds the identity of the page in virtual memory
    - It's a tag store!

Virtual Memory vs. Caching, cont.

- **Address view of virtual memory systems:**



- **In summary, virtual memory = software-managed fully-associative cache**

Virtual memory term	Caching term
Page number	Tag
Page offset	Offset
Page frame	*n/a*
n/a	Index
Page fault	Miss
Page table	*n/a*
“Inverted page table”	Tag store

NOW BACK TO CACHES…

Kinds of misses are…

- Compulsory misses
  - To have something in the cache, first it must be fetched
  - The initial fetch of anything is a miss
  - Also called unique references or first-time references
- Capacity misses
  - A miss that occurs due to the limited capacity in a fully-associative cache
  - The block/data was replaced before it was re-referenced
- Conflict misses
  - For set associative or direct mapped only
  - Misses due to the index bits matching
  - Also called mapping misses.
  - A conflict miss = misses that occur in a set-associative/DM cache that would not occur in a fully-associative cache of the same size

CACHE-32

Improving cache performance

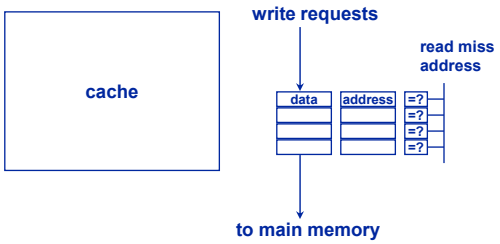
- Guide is: AAT = Hit time + Miss rate x Miss Penalty
  - Attack each term!
- Reduce miss rate
  - Block size, cache size, associativity, Victim cache
  - Prefetching: Hardware, Software
  - Data layout: Instructions, Data
- Reduce miss penalty
  - Write buffers
  - L2 caches
  - Subblocking
  - Early restart
  - Critical word first
- Reduce hit time
  - Avoid address translation (TLB accesses in parallel)
  - Simple caches, small caches
  - Pipeline writes
- Many other techniques possible
- Let's survey some of them

CACHE-33

Miss Penalty Reduction

Reduce miss penalty: write buffer

- Cache does not have to wait for writes to complete writing to memory
- Called a load/store buffer when it is used before the cache (see section on hit time reduction)
- For writes (write through) or writebacks (write-back):
  - Buffer accelerates subsequent read misses
  - On a read miss: (1) check for match (newest to oldest priority), (2) if no match, go to memory



Write buffers can cause problems

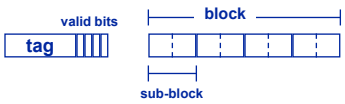
- We will see this when we discuss memory consistency
- Most processors allow the write buffer to be flushed:
  - Called a “memory fence”
- Some processors allow regions where the write buffer is completely turned off
  - Via “turn buffer off/on” instructions

TAG store sizes vs. Block sizes

- Small blocks lead to lower miss penalty
- BUT, large blocks need less tag storage:

- Assume address length = 64bits
- Also need 1 valid bit per tag
  - When the computer starts up, all blocks are marked as “not valid”
  - (You would also need to add 1 dirty bit per block for a WBWA cache, not shown below)
- Tag store in bits = $(64-(C-S)+1)*2^{C-B}$
  - Halving the block size doubles the tag store size!
- BUT small blocks have less spatial locality

Subblocking



- Problem: Tags are overhead, they take up extra space, but small blocks are desirable for low Miss Penalty
- Complete Solution:
  - Use large blocks, but also divide blocks into smaller "subblocks"
  - Fetch only 1 subblock on a miss *and* Keep valid bits for subblocks
  - Better if other subblocks are prefetched in the background
- If 2^k = subblock size, then need $v = 2^B / 2^k$ valid bits per block
- New tag store size in bits is $(64 – (C-S)+v)*2^{C-B}$
  - That's much less than if the block size were 2^k (i.e., $(64 – C+S+1)*2^{C-k}$)
- What about spatial locality?
  - On a miss to a subblock, *prefetch* to the end of the block (called 'load forwarding' because you're *loading forward* the rest of the sub blocks to the end of the block)

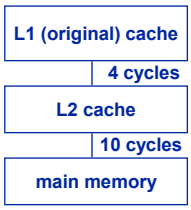
Early restart/Critical word first

- Problem: CPU must wait for missing data to be placed back into the cache
- Early Restart:
  - As soon as the requested word arrives, forward it to CPU
  - Miss penalty is now time to fetch the requested word
- Critical Word First
  - Problem still with Early Restart: What if requested word is in the middle of the block?
  - Start fetching the block with the required (critical) word, and fill in the rest
- Almost all L1 caches do both of these

Miss penalty reduction via L2 caches

- If it takes 10 cycles to go to main memory, why not put a cache between the L1 cache and main memory?

$$\text{Average access time} = \text{Hit time}_{L1} + \text{Miss rate}_{L1} \times \text{Miss penalty}_{L1}$$
$$\text{Miss penalty}_{L1} = \text{Hit time}_{L2} + \text{Miss rate}_{L2} \times \text{Miss penalty}_{L2}$$



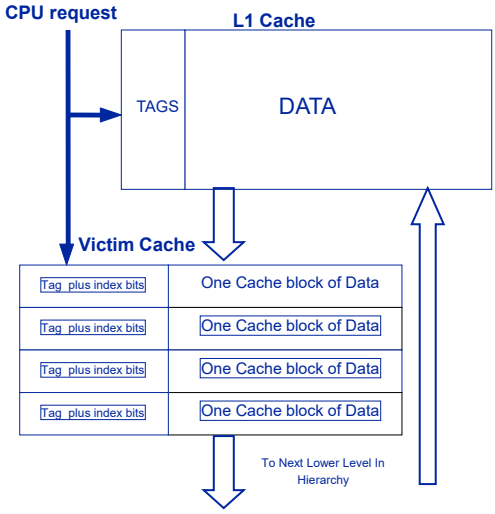
$$\text{Hit time}_{L1} = 2 \text{ cycles}$$
$$\text{Miss rate}_{L1} = 0.05$$
$$\text{Hit time}_{L2} = 4 \text{ cycles}$$
$$\text{Miss rate}_{L2} = 0.01$$
$$\text{Miss penalty}_{L2} = 10 \text{ cycles}$$
$$\text{Miss penalty}_{L1} = 4 + 0.01 \times 10 = 4.1 \text{ cycles}$$
$$\text{Average access time}_{L1} = 2 + 0.05 \times 4.1$$
$$= 2.21 \text{ cycles (w / o L2: 2.5 cycles)}$$

CACHE-40

40

Reduce penalty rate: Victim Cache

- Add buffer to hold data discarded from cache (the "victim")
- Swap blocks if found in VC and not in L1 cache
- Jouppi [1990]: 4-entry victim cache removed 20% to 95% of conflicts for a 4 KB direct mapped data cache
- "Extra associativity to rent" to each set



CACHE-41

41

AAT for a victim cache

- Because a VC is checked in parallel with the main cache:

$$AAT = HT + MR1 * MRvc * MP$$

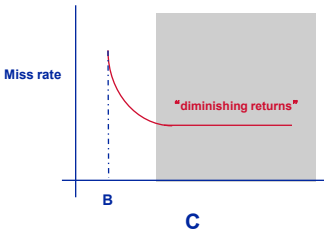
where MR1 = #cache_misses / #accesses

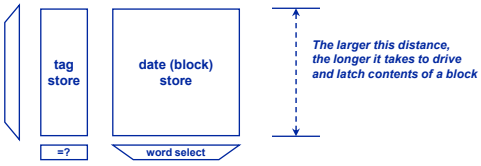
and MRvc = #vc_misses / #cache_misses

Miss Rate Reduction

Reduce miss rate: Cache size

- Advantages:
  - Larger caches hold more
- Disadvantages:
  - Steals resources from other units (esp. for on-chip caches)
  - Diminishing returns: double size != double performance
    - Limit is compulsory misses (also called the intrinsic miss ratio)
  - Larger caches are slower to access

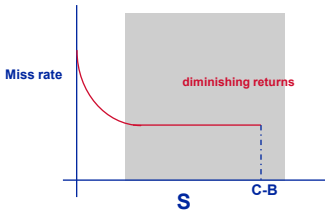




CACHE-44

Reduce miss rate: Inc. assoc.

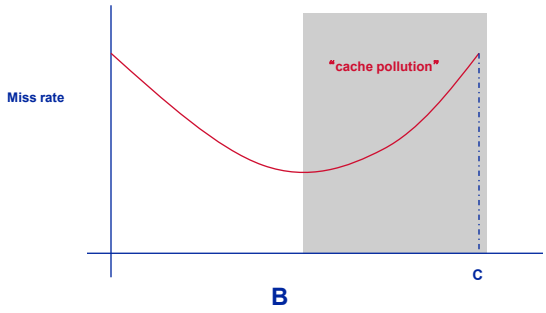
- Increase associativity
  - Advantages:
    - Fully associative performance is better than direct mapped
  - Disadvantages:
    - Slower (searching sets)
    - Diminishing returns
      - 8-way set associative is almost equivalent to fully associative in most cases
      - Asymptote is capacity misses



CACHE-45

Reduce miss rate: Block size

- Increase block size
  - Idea: exploit spatial locality
  - Problems:
    - don't overdo it: cache pollution from useless data
    - also increases miss penalty (have to bring more in)



Reduce miss rate: Prefetch

- Idea: get it *before* you need it
- Implementation:
  - +1 prefetch: fetch missing block, and next sequential block
    - Works great for streams with high sequential locality -- e.g., instruction caches
    - Uses unused memory bandwidth between misses
      - Can affect MP if bus to memory is busy and there's a demand miss by the CPU
  - Strided prefetch of degree *d*:
    - Idea memory is being accessed every *k* locations, so prefetch block(s) *+k, +2k, +3k…+d*k*:
    - If miss to A, then B, then C, prefetch C+k, where k = (B-A)

      example of code that has this behavior:

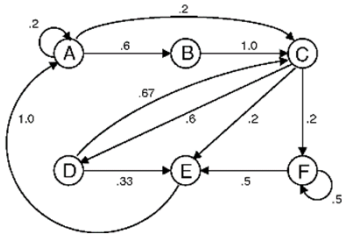
      ```
      for (i = 1; i < MAX; i += k)
          a[i] = b[i];
      ```

Prefetch performance aspects

- Over-prefetch and you can "kick out" useful data– cache polution
- Want prefetches to be accurate
  - If not being used, you should stop!
- Prefetches can "hog" memory bus
  - Solution: demand fetches get priority on the bus (called "abandonable" prefetch)
- Prefetches might arrive too late
  - Want to prefetch enough ahead to get *timeliness*

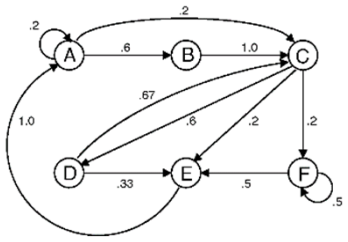
Complex predictor: Markov Prefetchers

- Work on the miss path stream
  - This is because it is difficult to "get inside" the CPU in higher levels of the memory hierarchy
- A,B,C,D,C,E,A,C,F,F,E,A,A,B,C,D,E,A,B,C,D,C
- Would produce this Markov chain:



From: *JOSEPH AND GRUNWALD: PREFETCHING USING MARKOV PREDICTORS, IEEE TRANSACTIONS ON COMPUTERS, VOL. 48, NO. 2, FEBRUARY 1999*

Markov predictor table for example

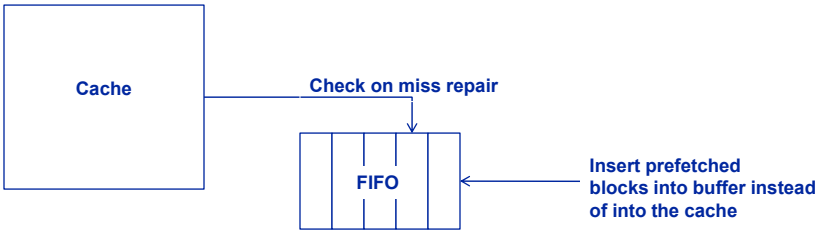


A,B,C,D,C,E,A,C,F,F,E,A,A,B,C,D,E,A,B,C,D,C

Each entry in a row is replaced by LFU when it fills up.
When the table fills up, rows are replaced via LRU

A	(B, 3)	(A, 1)	(C, 1)	
B	(C, 3)			
C	(D, 3)	(F, 1)	(E, 1)	
D	(C, 2)	(E, 1)		
E	(A, 3)			
F	(E, 1)	(F, 1)		

"Stream Buffers" help reduce cache pollution



- Prefetches go in a buffer that is checked on a miss
- This way, prefetches do not pollute the cache
- If found, "promote" these blocks to main cache and delete from buffer

Option 2: Instead of stream buffer, can also insert prefetches into the cache as the LRU (instead of MRU) block to reduce pollution

MISS RATE REDUCTION (cont.): Compiler optimizations

Compiler directed prefetch:

- Need "nonbinding prefetch" inst., like a load, but:
  - Doesn't cause a page fault
  - Doesn't change processor's state
  - Doesn't make processor wait for a prefetch miss
- Compiler estimates which accesses miss
- Inserts prefetch instructions well enough ahead to prevent the disaster of a cache miss
- Reduces ***compulsory*** misses (the compulsory misses simply move around, since the prefetch instructions still generate the misses)

```
for (j = 0; j < 100; j++)
   for (i = 0; i < 100; i++)
      x[i][j] = c * x[i][j]
```



```
for (j = 0; j < 100; j++)
   for (i = 0; i < 100; i++) {
      prefetch(x[i][j+k]);
      x[i][j] = c * x[i][j];
   }
```

Where *k* depends on the miss penalty and the time it takes to execute an iteration

Layout for Data via Merging

- Lay out arrays that are accessed together in the same array (i.e., interleaved in memory)

Code:

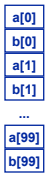
```
int a[100];
int b[100];

for (i = 0; i < MAX; i += 2)
   a[i] = b[i];
```

Unoptimized:



Compiler optimized:



What if block size = two "int"s? Each access to a[i] or b[i] brings in a[i+1] and b[i+1], which are never used

Now every access to a missing block brings in useful data as well (we've enhanced the spatial locality of the code).

Problem: DANGEROUS because some programmers assume a specific layout! e.g., *(a+1) == a[1] in C, but now == b[0]

Other optimizations (cont.):

- Loop Fusion
  - Two loops with identical sets of iterations can be combined:

```
for (j = 0; j < MAXJ; j++)
   for (i = 0; i < MAXI; i++)
      a[i][j] = cos(x[i][j]);

// other stuff

 for (j = 0; j < MAXJ; j++)
    for (i = 0; i < MAXI; i++)
       z[i][j] = a[i][j] * d[i];
```

⇩

```
for (j = 0; j < MAXJ; j++)
   for (i = 0; i < MAXI; i++) {
      a[i][j] = cos(x[i][j]);
      z[i][j] = a[i][j] * d[i];
   }
```

problem is that a[i][j] is written entirely in the first loop before it can be read in the second loop, reducing the ***chance*** that it is still in the cache (a guess on the part of the compiler)

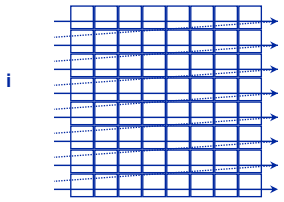
problem reduced by "fusing" the two loops

Other optimizations (cont.):

- Tiling (also called, confusingly, "blocking")

  Idea: access "regions" of arrays instead of whole array

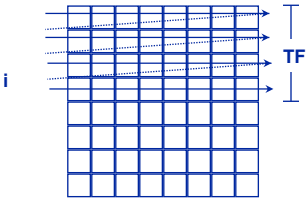
Each value of k:




```
for (k = 0; k < N; k++)
  for (i = 0; i < N; i++)
    for (j = 0; j < N; j++)
      sum += x[i][j]*(k+1);
```

problem: *not all of x[i][j] can fit in the cache*

for (k = 0; k < N; k++)




```
for (ii = 0; ii < N; ii += TF)
  for (k = 0; k < N; k++)
    for (i = ii; i < ii+TF; i++)
      for (j = 0; j < N; j++)
        sum += x[i][j]*(k+1);
```

Tiling factors "TF" selected so that a "tile" of x[i][j] fits in the cache

Now only compulsory misses for x[i][j]

Other optimizations:

- Loop interchange
  - Increase temporal locality by exchanging inner and outer loops

```
for (k = 0; k < N; k++)
  for (i = 0; i < N; i++)
    for (j = 0; j < N; j++)
      sum += x[i][j]*(k+1);
```

⇩

```
for (i = 0; i < N; i += 1)
  for (j = 0; j < N; j += 1)
    for (k = 0; k < N; k++)
      sum += x[i][j]*(k+1);
```

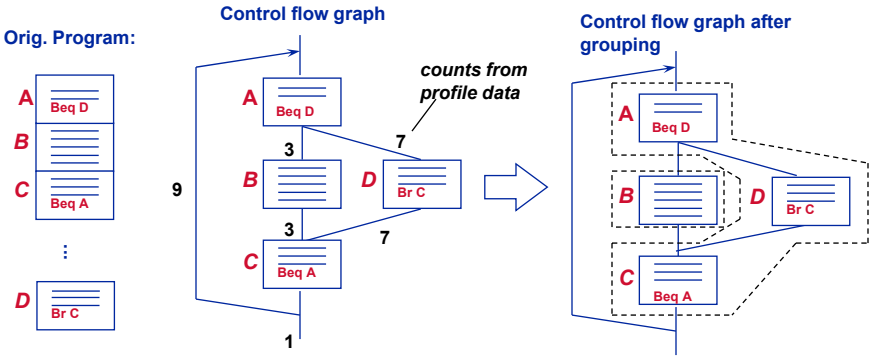
Note: cannot vectorize inner most loop now

This is why tiling is used instead of loop interchange is

Compiler layout of inst./data

- Idea: if fetch A then B, put A & B in the same block (spatial locality)
- For instruction caches:
  - For most instructions, this happens normally
  - Branches change the sequential access pattern
  - Solution:
    - Figure out how frequently every branch is taken/not-taken
    - Form control flow graph
      - Draw boxes around largest sequential runs of code (see example on next page)
    - Find groups of instructions that tend to execute one after another
    - Rewrite program putting these instructions into sequential order

Instruction layout



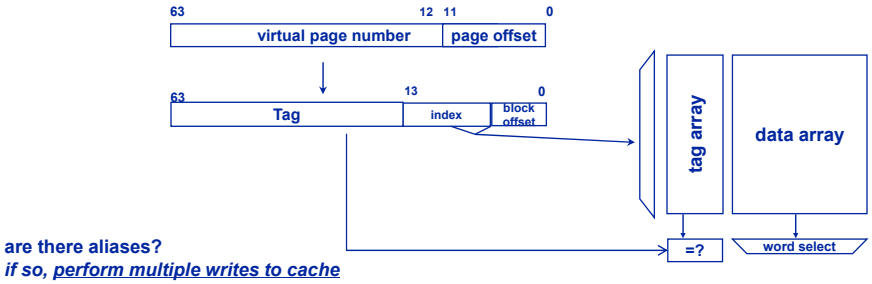
- Write out program:
  - Instructions in this order: A, D, C, then B

Hit Time Reduction

Hit time reduction techniques

- Direct-mapped caches
  - No need for search of members of the set
  - Might require larger caches for equal miss rate
  - Sometimes page size doesn't let you do it
- Small caches
  - (See discussion of cache size)
- Load/Store buffer
  - Similar to a write buffer (see section on miss penalty reduction)
  - Difference: Accessed before the cache instead of at miss time

Virtually indexed, Virtually Tagged cache



are there aliases?
if so, perform multiple writes to cache

- BIG problem, though:
  - Modern OS's implement shared memory between processes or threads by assigning two (or more) virtual addresses the same physical page frame. This is called an "alias"
  - So every write needs to check a Reverse Translation Buffer and write to *all blocks* that have the same page frame

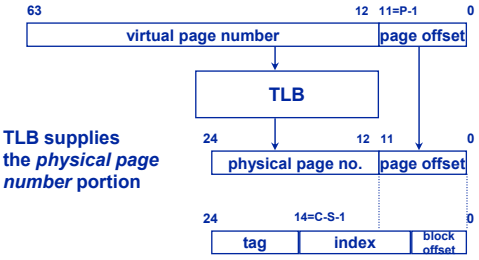
Virtually vs. physically addressed caches

- The CPU core generates virtual addresses, which correspond to locations in virtual memory
- A special cache (the translation lookaside buffer) is checked to find the mapping of the virtual address to physical address
- On a miss, the cache must fetch a block from the physical memory



Problem: this increases the hit time! (e.g., from one to two cycles)

Specifics of virtual address translation



TLB supplies the *physical page number* portion

Ex: page size = 4KB
page offset bits = $\log_2$(4KB) = 12

say 32MB of physical memory (25 bits of physical address)

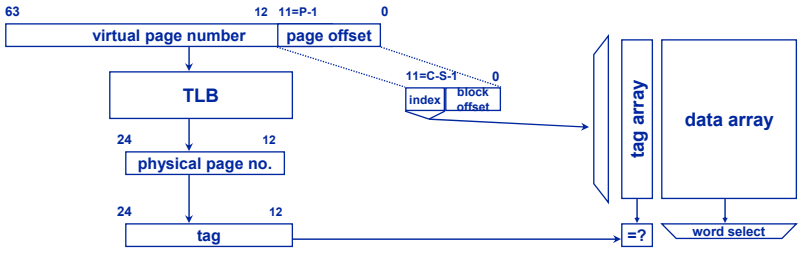
This is how cache interprets the physical address

- Observation: What if *index* bits were entirely contained in page offset bits?
  - Then 1st part of cache access would not wait on TLB

CACHE-67

67

Virtually indexed, physically tagged caches



- Cache hit time reduces from 3 cycles to 2!
  - Because cache can now be *indexed* in parallel with TLB (tag match uses output from TLB)
  - But some constraints...

CACHE-68

68

Page size/associativity tradeoffs

- **Options:**
  - For new machines: Select page size such that:

$$P \geq C - S$$

  - When page size is fixed: Select assoc. such that:

$$S \geq C - P$$

- **Example: MC88110**
  - page size = 4KB
  - I-cache, D-cache are both: 8KB, 2-way set-assoc.
  - 4KB = (8KB / 2)
- **Example: VAX series**
  - page size = 512B
  - For a 16KB cache, need assoc. = (16KB / 512B) = 32-way set. assoc. !
- **Moral: Sometimes associativity is thrust upon you!**